Lecture 9

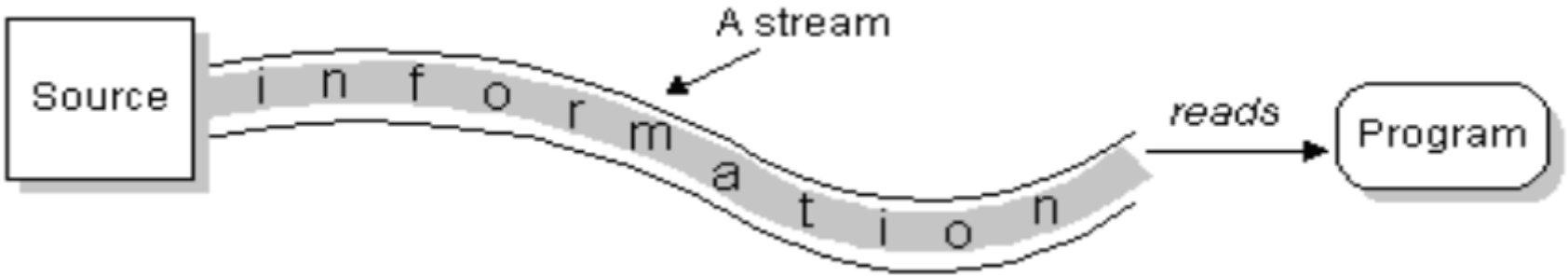
Input/Output (I/O)

Introduction to data streams

- Data input/output activities are all referred to as streams
  - Reading data from keyboard, files
  - Writing data to the screen, writing data to disk (as files), printing to a printer...
- All streams operate on *the same principle* even when they are associated with different physical devices

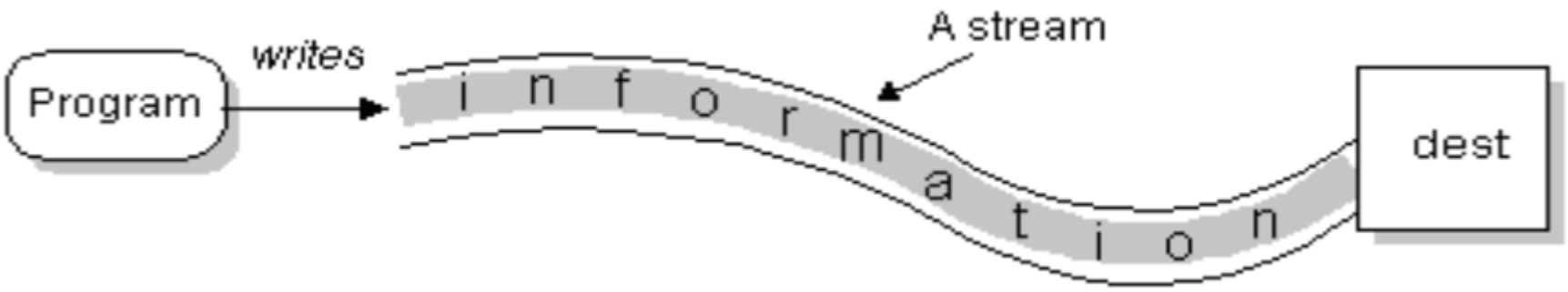
Introduction of Data Streams

- **Input stream** is a stream that allows the program to read data from a source such as keyboard, file, scanner...



Introduction of Data Streams

- **Output stream** is a stream that allows the program to write data to it to be sent to a destination such as screen, file, printer...



Visual summary of Data streams

Reading

Writing

Two main types of streams

- Byte Streams (binary)
  - Work with raw bytes
  - Used for images, videos, binary files
- Character Streams (text)
  - Work with Unicode characters
  - Used for text files
- Key difference:
  - Byte = low-level
  - Character = human-readable

When to use which?

Situation	Use
Text file	Character stream
Binary file	Byte stream
Structured primitive data	Data streams
Java objects	Object streams

Motivating Problem

Student Management System

We want to:

1. Save student data to file
2. Load it back later
3. Support different formats:
   - Text (human-readable)
   - Binary (efficient)
   - Object (Java objects)

➢ ***How should we store data?***

1st solution – File reading/writing (Text)

```
FileWriter writer = new FileWriter("students.txt");
writer.write("Alice\nBob\n");
writer.close();
```

```
FileReader reader = new FileReader("students.txt");
int ch;
while ((ch = reader.read()) != -1) {
    System.out.print((char) ch);
}
reader.close();
```

- Simple but not efficient!
  - Stored data is human-readable, but not machine-readable
  - No support for data structure

2nd solution: Data streams

```
DataOutputStream dos = new DataOutputStream(
        new FileOutputStream("data.dat"));
dos.writeInt(3);
dos.writeDouble(9.5);
dos.writeUTF("Hello");
dos.close();
```

```
DataInputStream dis = new DataInputStream(
        new FileInputStream("data.dat"));
int n = dis.readInt();
double d = dis.readDouble();
String s = dis.readUTF();
dis.close();
```

- Supports primitive types & String
  - Stored data is human-readable
  - Still not convenient (have to write code to read data correctly)

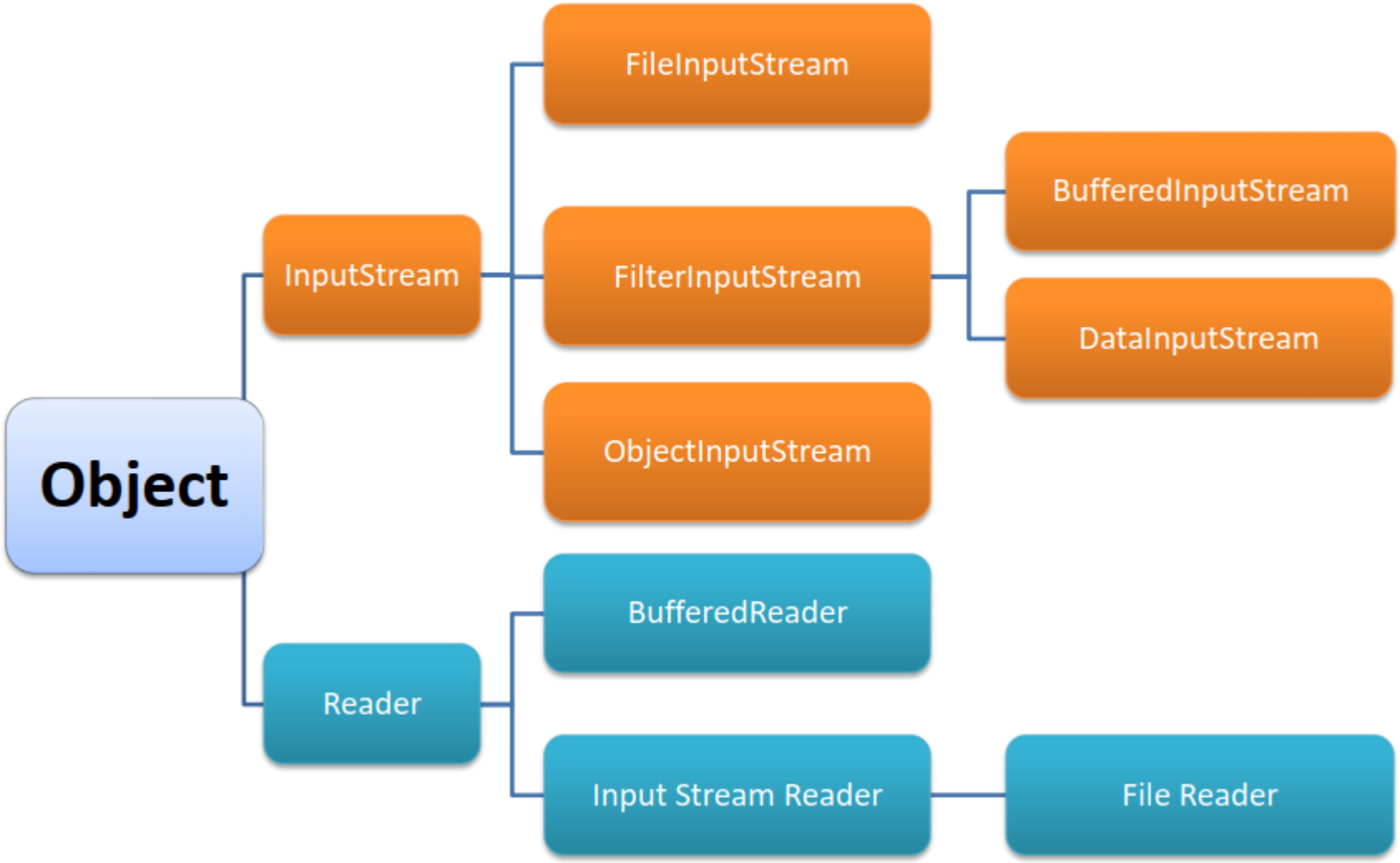
3rd solution: Object streams

```
ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream("obj.dat"));
oos.writeObject(student);
oos.close();
```

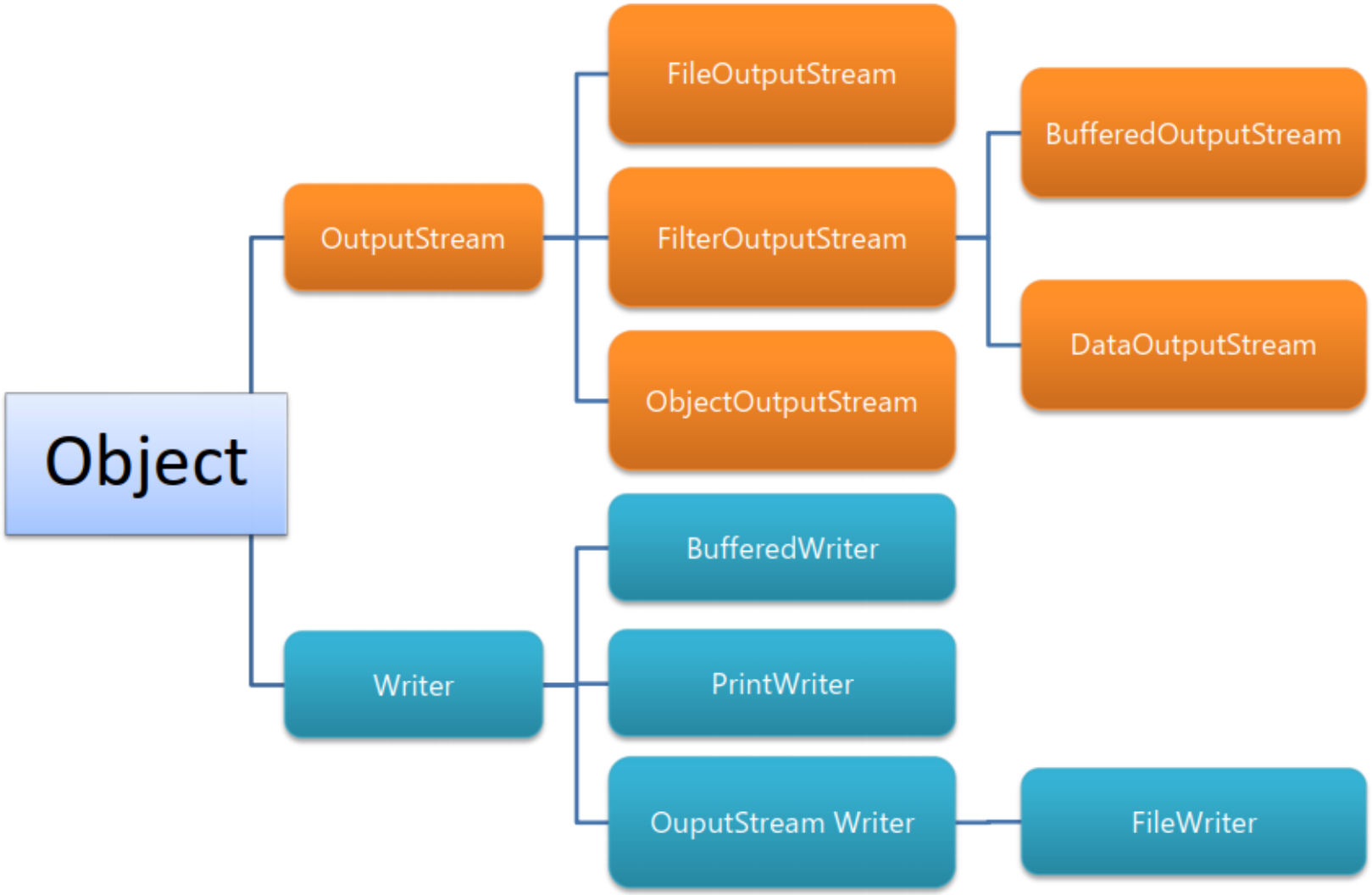
```
ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream("obj.dat"));
Student s = (Student) ois.readObject();
ois.close();
```

- Supports structured data: save & read whole objects
  - Not human-readable
  - No need to write reading/writing logic (convenient)
  - Requires Serializable (`class Student implements Serializable`)

Inheritance hierarchy of Input Streams



Inheritance hierarchy of Output Streams



Byte streams

- Using byte streams for binary data input/output
- All byte streams inherit from 2 classes:
  - `InputStream`
  - `OutputStream`
- There are many byte stream classes:
  - `FileInputStream`
  - `FileOutputStream`
- They differ in initialization methods but operate in the same way.

`FileInputStream` / `FileOutputStream`

- These streams are used to work with binary files.
  - Use `FileInputStream` to read data from binary files.
  - Use `FileOutputStream` to write data to binary files.

FileOutputStream

- `FileOutputStream` is an output stream used to write data to binary files.

```
FileOutputStream fos = new FileOutputStream("file1.txt");
String txt = "Tôi là sinh viên xuất sắc của FIT.";
byte[] data = txt.getBytes(StandardCharsets.UTF_8);
System.out.println("String length: " + txt.length());
System.out.println("Byte array length: " + data.length);
fos.write(data);
fos.close();
```

FileInputStream

- `FileInputStream` is the input stream for reading data from binary files.

```
public class Example2 {
    public static void main(String[] args) throws IOException {
        FileInputStream fis = new FileInputStream("file1.dat");
        int c;
        while ((c = fis.read()) != -1) {
            System.out.print((char) c);
        }
        fis.close();
    }
}
```

FileInputStream

```
FileInputStream fis = new FileInputStream("file1.txt");
byte[] buffer = new byte[100];
int bytesRead = fis.read(buffer);
System.out.println("We've read " + bytesRead + " bytes.");
byte[] data = new byte[bytesRead];
String text = new String(buffer, 0, bytesRead, UTF_8);
System.out.println("File content: " + text);
fis.close();
```

DataInputStream / DataOutputStream

- These two streams help us read/write primitive data.
- Use `read<Type>()` to read primitive data from `DataInputStream`.
  - `readInt()`
  - `readDouble()`
  - …
- Use `write<Type>(Type)` to write primitive data to `DataOutputStream`.
  - `writeBoolean(boolean)`
  - `writeInt(int)`
  - …

DataOutputStream

```java
public class DataStreamOutput {
    public static void main(String[] args) throws IOException {
        FileOutputStream fos = new FileOutputStream("filedata.dat");
        DataOutputStream dos = new DataOutputStream(fos);
        final int NUMBER = 5;
        dos.writeInt(NUMBER);
        for (int i = 0; i < NUMBER; i++) {
            dos.writeInt(i);
        }
        dos.writeUTF("Hello!");
        dos.writeDouble(100.75);
        dos.flush();
        dos.close();
    }
}
```

DataInputStream

```
public class DataStreamInput {
    public static void main(String[] args) throws IOException {
        FileInputStream fis = new FileInputStream("filedata.dat");
        DataInputStream dis = new DataInputStream(fis);
        int items = dis.readInt();
        for (int i = 0; i < items; i++) {
            int n = dis.readInt();
            System.out.print(n + " ");
        }
        System.out.println(dis.readUTF());
        System.out.println(dis.readDouble());
        dis.close();
    }
}
```

`ObjectInputStream` / `ObjectOutputStream`

- This stream helps us read/write objects.
- Use `readObject()` to read objects from `ObjectInputStream`.
- Use `writeObject(Serializable)` to write objects to `ObjectOutputStream`.
- **Note:**
  - Only objects created from classes implementing the `Serializable` interface can be read/written.

Serializable interface

```
public class Stock implements Serializable {
    private int id;
    private String desc;
    private double price;
    private int quantity;

    public Stock(int id, String desc, double price, int quantity) {
        this.id = id;
        this.desc = desc;
        this.price = price;
        this.quantity = quantity;
    }

    public String toString() {
        return (id + " " + desc + " " + price + " " + quantity);
    }
}
```

ObjectOutputStream

```
public class ExampleWriteObject {
    public static void main(String[] args)
            throws IOException, ClassNotFoundException {
        FileOutputStream fos = new FileOutputStream("stocks.dat");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        Stock[] stocks = {
                new Stock(1001, "CD ROM", 100.00, 20),
                new Stock(1002, "DRAM", 75.00, 30),
                new Stock(1003, "P4 Processor", 300.00, 100),
                new Stock(1004, "Canon Jet", 80.00, 10),
                new Stock(1005, "HP Scanner", 75.00, 90)
        };
        // write the object array into 'stocks.dat'
        oos.writeObject(stocks);
        oos.close();
    }
}
```

ObjectInputStream

```java
public class ExampleReadObject {
    public static void main(String[] args) {
        FileInputStream fis = null;
        ObjectInputStream ois = null;
        try {
            fis = new FileInputStream("stocks.dat");
            ois = new ObjectInputStream(fis);
            Stock[] stocks = (Stock[]) ois.readObject();
            System.out.println("Read from file: ");
            for (Stock s : stocks) {
                System.out.println(s);
            }
            ois.close();
            fis.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Character Streams

- *Byte streams* are powerful and flexible.
  - However, *character streams* operate directly on Unicode characters.
- All character streams are inherited from 2 abstract classes: `Reader` and `Writer`.

Java default character set

- Use this code to find out the current default character set used by Java
  - **Note:** From Java 17+, the following is preferred: `Charset.defaultCharset().name())`

```
public class DefaultCharset {
    public static void main(String[] args) {
        String chSet = System.getProperty("file.encoding");
        System.out.println(chSet);
    }
}
```

FileWriter / FileReader

```java
File filename = new File("first.txt");
try {
    // writing
    FileWriter out = new FileWriter(filename);
    out.write("Write something");
    out.write("\n");
    out.write("Character Streams");
    out.close();
    // reading
    FileReader input = new FileReader(filename);
    System.out.println("Read from first.txt");
    int ch = input.read();
    while (ch != -1) {
        System.out.println((char) ch);
        ch = input.read();
    }
} catch (IOException e) {
    throw new RuntimeException(e);
}
```

`BufferedReader` / `BufferedWriter`

- This stream is used to work with buffered character streams.
- Use `BufferedReader` to work with buffered input text streams.
- Use `BufferedWriter` to work with buffered output text streams.

BufferedWriter

```
public class StudentManager {
    private Student[] students;
    private int count;

    public void writeToTextFile(String fileName) throws IOException {
        FileWriter fw = new FileWriter(fileName);
        BufferedWriter bw = new BufferedWriter(fw);
        for (int i = 0; i < this.count; i++) {
            String temp = students[i].toString();
            bw.write(temp);
        }
        bw.flush();
        bw.close();
    }
}
```

BufferedReader

```
public void readFromTextFile(String FileName)
        throws IOException, ClassNotFoundException {
    FileReader fr = new FileReader(FileName);
    BufferedReader br = new BufferedReader(fr);
    String text;
    while ((text = br.readLine()) != null) {
        System.out.println(text);
    }
    br.close();
}
```

Input/Output Exception Handling

- When input/outputting data, there are *checked* exceptions that must be caught when writing code, usually these exceptions are:
  - `FileNotFoundException`
  - `EOFException`
  - `NotSerializableException`
  - `IOException`

Using `try…catch` in Input/Output

```
FileOutputStream fos = null;
try {
    fos = new FileOutputStream("file1.txt");
    String text = "The quick brown fox jumped over the lazy dog";
    byte[] textAsBytes = text.getBytes();
    fos.write(textAsBytes);

} catch (FileNotFoundException e) {
    throw new RuntimeException(e);
} catch (IOException e) {
    throw new RuntimeException(e);
} finally {
    try {
        fos.close();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

try-with-resources (Java 7+)

```
// try-with-resources: streams are closed automatically
try (FileOutputStream fos = new
FileOutputStream("data.dat");
     DataOutputStream dos = new DataOutputStream(fos)) {
    dos.writeInt(42);
    dos.writeUTF("Hello FIT!");
    // No need to call close() — done automatically
} catch (IOException e) {
    e.printStackTrace();
}
```

Reading files with Scanner

- Scanner is the most popular way to read text files in practice

```
try (Scanner sc = new Scanner(new File("students.txt"))) {
    while (sc.hasNextLine()) {
        String line = sc.nextLine();
        String[] parts = line.split(";");
        int id = Integer.parseInt(parts[0]);
        String name = parts[1];
        System.out.println(id + " - " + name);
    }
} catch (FileNotFoundException e) {
    e.printStackTrace();
}
```

Writing files with `PrintWriter`

- More powerful & easier to use than `FileWriter`

```
try (PrintWriter pw = new PrintWriter(
        new FileWriter("output.txt", true)
    )) {
    // true = append mode
    pw.println("Student ID: 1001");
    pw.println("Name: Nguyen Van A");
    pw.printf("GPA: %.2f%n", 3.75);
    pw.flush();
} catch (IOException e) {
    e.printStackTrace();
}
```

Serializable: `serialVersionUID` & transient

```java
public class Student implements Serializable {
    // Declare serialVersionUID explicitly
    // to avoid InvalidClassException
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;
    private double gpa;
    // transient: field is NOT included in serialization
    private transient String password;
    public Student(int id, String name, double gpa) {
        this.id = id;
        this.name = name;
        this.gpa = gpa;
    }
}
```

Summary

- Data Streams
- Input/Output Streams Hierarchy
- Byte Streams
  - `FileInputStream/FileOutputStream`
  - `DataInputStream/DataOutputStream`
  - `ObjectInputStream/ObjectOutputStream`
- Character Streams
  - `FileWriter/FileReader`
  - `BufferedReader/BufferedWriter`
- Using try…catch in Input/Output
  - `try-with-resources` (Java 7+)
- Additional Classes
  - `Scanner` / `PrintWriter`
  - `serialVersionUID` and `transient`